

Common Foundations for SHACL, ShEx, and PG-Schema

Shqiponja Ahmetaj (TU Wien) Iovka Boneva (U Lille) Jan Hidders (Birkbeck, U London)
Katja Hose (TU Wien) Maxime Jakubowski (TU Wien) Jose Emilio Labra Gayo (U Oviedo)
Wim Martens (U Bayreuth) Fabio Mogavero (U Napoli) Filip Murlak (U Warsaw)
Cem Okulmus (U Paderborn) Axel Polleres (WU Wien, CSH Vienna) Ognjen Savkovic (FU Bolzano)
Mantas Simkus (TU Wien) Dominik Tomaszuk (U Bialystok, TU Wien)

TheWebConf 2025 @ Sydney, Australia

It's a Wild Wild West

There are **large knowledge graphs** out there:

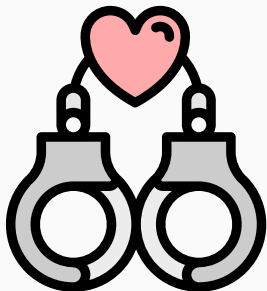
- DBPedia, Wikidata, Yago, Google Knowledge Graph, Amazon product graph

More and more realisation that **knowledge graphs need schemas**:

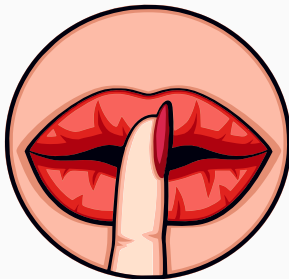
- tell users and systems what to expect in the data, limit data modifications;
- for data exploration, visualisation, integration, and curation, and query formulation and optimisation;
- ad-hoc solutions deployed, multiple standards emerging

Three schema languages for graphs

SHACL



ShEx



PG-Schema



Understand **differences** and **commonalities** between SHACL, ShEx & PG-Schema.

Users are asking for this.

- What are the fundamental differences? Roundtrip possibilities?

Vendors are asking for this.

- What is actually used? Can't we have just one?

Designers should be asking for this.

- Why so different solutions despite similar objectives?
- Is it by design? Do the differences matter?

SHACL



ShEx



PG-Schema



Data model

RDF

RDF

LPG

Paradigm

Description logic for specifying constraints

Regular expressions for neighbourhoods, close to XML-Schema

Simple types for nodes and edges, rich key constraints

Users

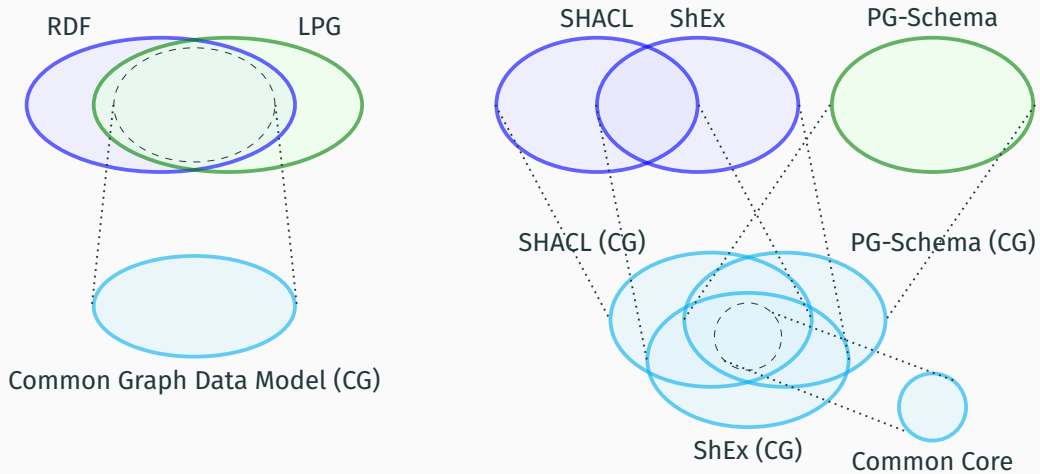
Data architects

Data architects

Programmers

*For if your goal be Paradise
[...]
Ponder all your wants in life
And make A Little Sacrifice...*

Method



Why?

Who knows all three:

- SHACL,
- ShEx,
- PG-Schema?

Why?

Who knows all three:

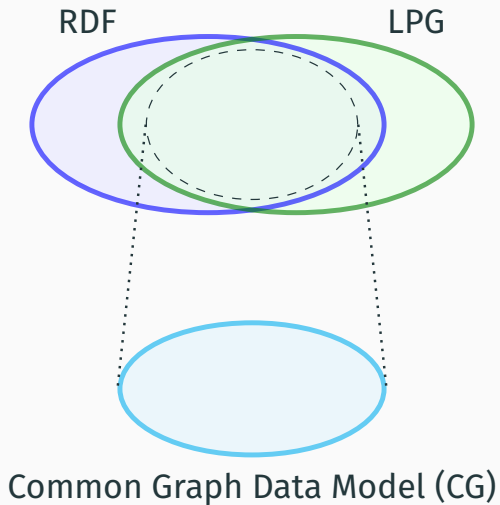
- SHACL,
- ShEx,
- PG-Schema?

We describe all three using a **common vocabulary** and **uniform syntax**:

- you can understand it using the language you already know,
- and you learn the other two languages for free.

1. Distilling a common data model

Distilling the Common Graph Data Model



Features not present in both RDF and LPG are removed.

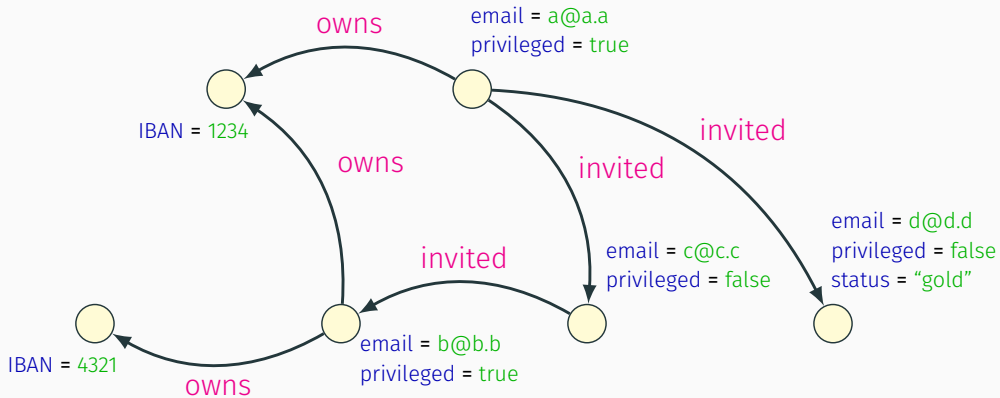
What RDF gives up:

- **Nodes** ...
 - cannot be compared with an IRI
- **Predicates** ...
 - associate at most one value with a node
 - cannot be compared with a node

What LPG gives up:

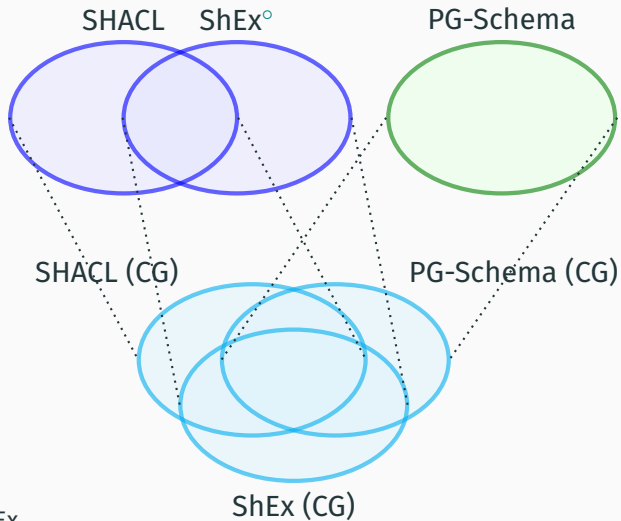
- **Nodes** ...
 - do not have labels
- **Edges** ...
 - have exactly one label
 - have no properties
 - are identified by label and endpoints

Common Graph Example



2. Distilling the schema languages

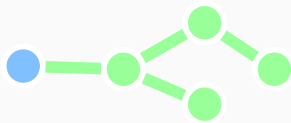
Distilling the three schema languages



[◦] Non-recursive ShEx

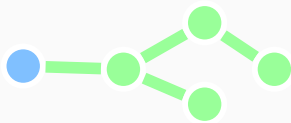
A common vocabulary to describe schema languages

A **shape** describes the graph's structure around a **focus** node or value.



A common vocabulary to describe schema languages

A **shape** describes the graph's structure around a **focus** node or value.

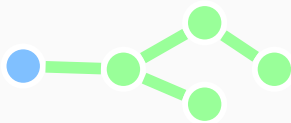


A **selector** is a simple shape that indicates **focus** nodes or values.



A common vocabulary to describe schema languages

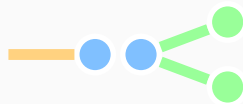
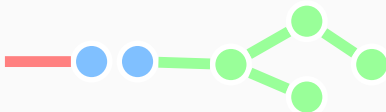
A **shape** describes the graph's structure around a **focus** node or value.



A **selector** is a simple shape that indicates **focus** nodes or values.

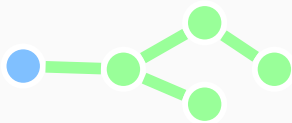


A **schema** is a finite set of selector-shape pairs.



A common vocabulary to describe schema languages

A **shape** describes the graph's structure around a **focus** node or value.



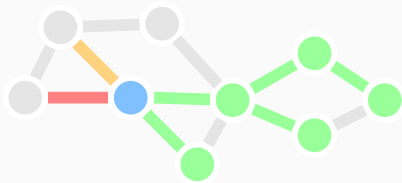
A **selector** is a simple shape that indicates **focus** nodes or values.



A **schema** is a finite set of selector-shape pairs.



A graph G **conforms to** schema $\mathcal{S}$ if,
for each selector-shape pair (sel, shp) in $\mathcal{S}$,
for every focus v indicated by sel ,
 G around v has shape described by shp .



The three distillates

SHACL: Gold status when network has 100 users whose networks have 100 users.

$$\exists \text{status}.\top \implies \exists \text{status}.\text{"gold"} \rightarrow \exists^{\geq 100} \text{invited}^*.\exists^{\geq 100} \text{invited}^*.\top$$

ShEx: Every user can invite (directly) at most 10% of privileged users.

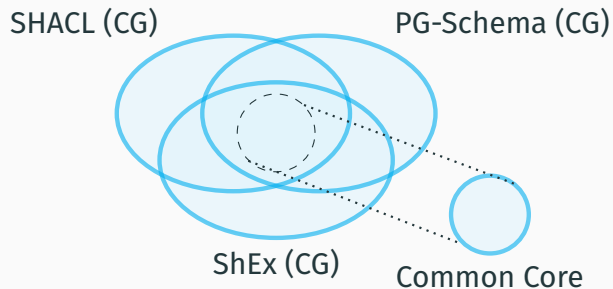
$$\{\text{status}.\{\top\}\}^\circ \implies \left\{ \left(\text{invited}.\{\text{privileged}.\text{false}\}^\circ \right)^* ; \right. \\ \left. \left(\text{invited}.\{\text{privileged}.\text{true}\}^\circ ; \left(\text{invited}.\{\text{privileged}.\text{false}\}^\circ \right)^9 \right)^* \right\}^\circ$$

PG-Schema: Platinum status when privileged network has 100 users.

$$[\text{status} = \text{"platinum"}] \implies \exists^{\geq 100} (\text{invited} \cdot [\text{privileged} = \text{true}])^*$$

3. Extracting the common core

Extracting the Common Core



Extracting the Common Core

SHACL and **ShEx** give up

- disjunctions and negations of shapes,
- nesting shapes.

$\exists \text{invited.T} \vee \neg \exists \text{status."gold"}$

$\exists^{\geq 100} \text{invited} . \exists^{\geq 100} \text{invited.T}$

Extracting the Common Core

SHACL and **ShEx** give up

- disjunctions and negations of shapes,
- nesting shapes.

$\exists \text{invited.T} \vee \neg \exists \text{status."gold"}$
 $\exists^{\geq 100} \text{invited} . \exists^{\geq 100} \text{invited.T}$

SHACL and **PG-Schema** give up

- unbounded path expressions (star),
- counting over multi-edge path expressions.

$\exists \text{invited}^* . \text{privileged.true}$
 $\exists^{\geq 100} \text{invited}^{\leq 3} . \text{T}$

Extracting the Common Core

SHACL and ShEx give up

- disjunctions and negations of shapes,
- nesting shapes.

$\exists \text{invited}.\top \vee \neg \exists \text{status}.\text{"gold"}$
 $\exists^{\geq 100} \text{invited}.\exists^{\geq 100} \text{invited}.\top$

SHACL and PG-Schema give up

- unbounded path expressions (star),
- counting over multi-edge path expressions.

$\exists \text{invited}^* \cdot \text{privileged}.\text{true}$
 $\exists^{\geq 100} \text{invited}^{\leq 3}.\top$

PG-Schema gives up

- closing properties and edges independently, $\{\text{IBAN} : \text{STRING}\} \wedge \{\text{owns}^-\}$
- closing the whole graph, $\top \implies \{\text{IBAN} : \text{STRING}\} \mid \{\text{email} : \text{STRING}\}$
- testing properties under star. $\exists (\text{invited} \cdot [\text{privileged} = \text{true}])^* \cdot \text{owns}$

Common Graph Schema Language (CoGSL)

Direct neighbourhood specified using counting constraints and node types.

$[\text{status} = \text{"gold"}] \implies \exists^{\geq 100} \text{invited} \cdot [\text{privileged} = \text{true}]$

$\exists \text{status} \implies \{\text{status} : \text{STRING}, \text{privileged} : \text{BOOL}, \text{email} : \text{STRING}, \text{invited}, \text{owns}\}$

Farther away, only presence or absence of paths (of bounded length).

$[\text{status} = \text{"founding father"}] \implies \exists (\text{invited} \cdot [\text{privileged} = \text{true}])^{10}$

$\exists \text{status} \implies \nexists \text{invited} \cdot \text{invited} \cdot [\text{status} = \text{"first generation"}]$

Conclusions

What have we gained?

A **common vocabulary** to talk about SHACL, ShEx, and PG-Schema, compare their core mechanisms and expressive power:

- a high-level notion of a shape-based graph schema;
- a uniform syntax for shapes and selectors.

A **foundation** for automating interoperability:

- a data model that can be supported by both RDF and LPG;
- a schema language that can be compiled to SHACL, ShEx, and PG-Schema.

What do we do next?

- Bring back recursion in ShEx (nearly there).
- Explore real-life schemas, evaluate coverage offered by CoGSL.
- Analyse expressive power and complexity of all three schema languages.
- Develop interoperability tools.